CI/CD & PYDANTIC

A practical quick-reference guide for data scientists

Two concepts. One goal: build Python applications and data workflows that are more reliable, repeatable and easier to operate.

What this guide covers

- **CI/CD:** what it means, how GitHub Actions fits in, and how a data science workflow moves from code changes to production.
- **Pydantic:** configuration and settings, validation, API inputs, and structured output from LLMs.
- **Interview readiness:** concise explanations you can adapt without overstating your hands-on experience.

At a glance

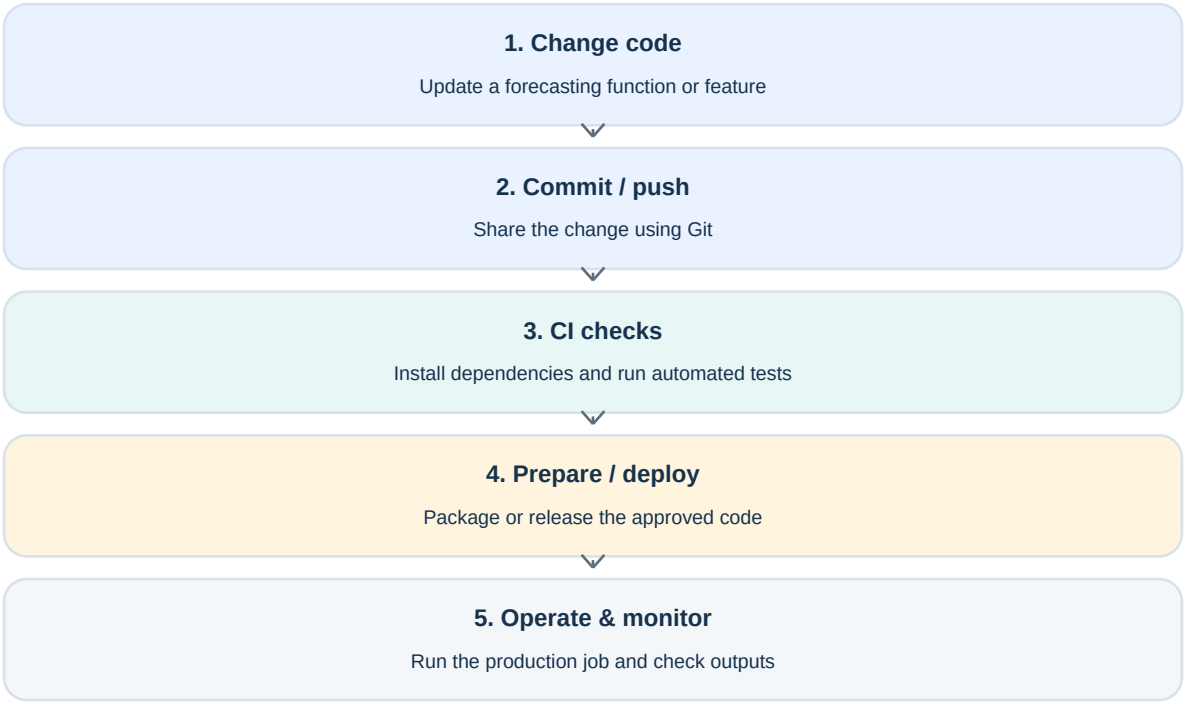
CI/CD helps automate testing and releasing code. **Pydantic** helps define and validate the structure of data moving through Python applications.

1. CI/CD: the high-level idea

CI/CD stands for **Continuous Integration** and **Continuous Delivery** or **Continuous Deployment**. It is a set of practices that automates the validation and release of code changes.

Term	Plain-English meaning	Example
CI — Continuous Integration	Automatically build and test code when changes are pushed or proposed.	Run unit tests after a pull request is opened.
Continuous Delivery	Keep validated code ready to release; production release may need approval.	A tested package is ready for an approved deployment.
Continuous Deployment	Automatically release validated changes to production when required checks pass.	A successful pipeline deploys the new application version.

A typical flow

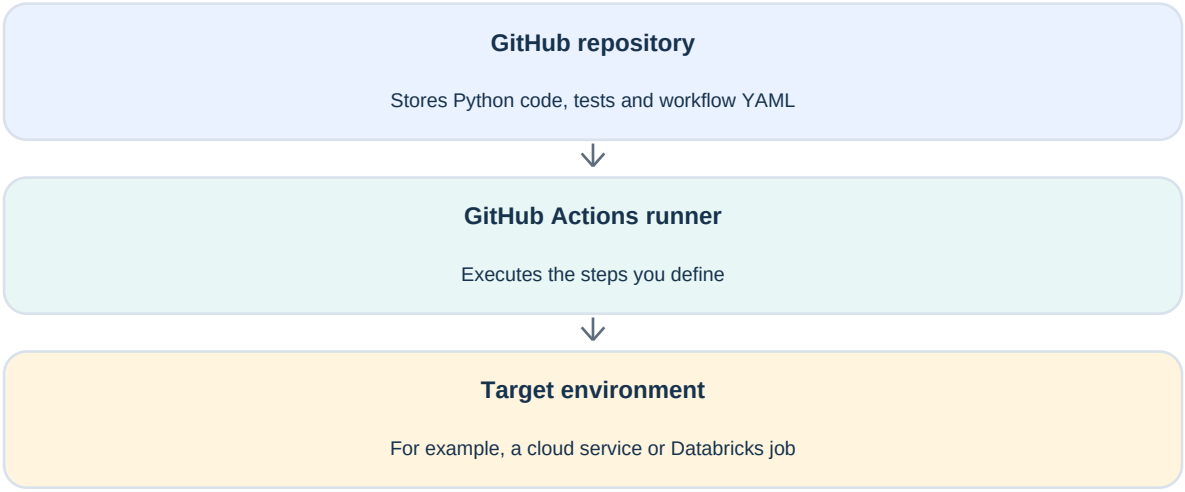


Key distinction

CI/CD is not just scheduling a script. The central idea is automating the checks and release process around code changes. Scheduling is operational automation and may be part of a wider workflow.

2. Where GitHub Actions fits

GitHub Actions is a workflow automation tool. You describe a workflow in a YAML file; GitHub runs the defined steps when a trigger occurs. It can run tests, build packages, publish artifacts, deploy applications, and schedule scripts.



Common triggers

Trigger	When it runs	Typical use
push	Code is pushed to a branch.	Run tests after a change.
pull_request	A pull request is opened or updated.	Check proposed changes before merging.
schedule	At a configured time.	Run a nightly data task.
workflow_dispatch	A user manually starts it.	Run a one-off workflow.

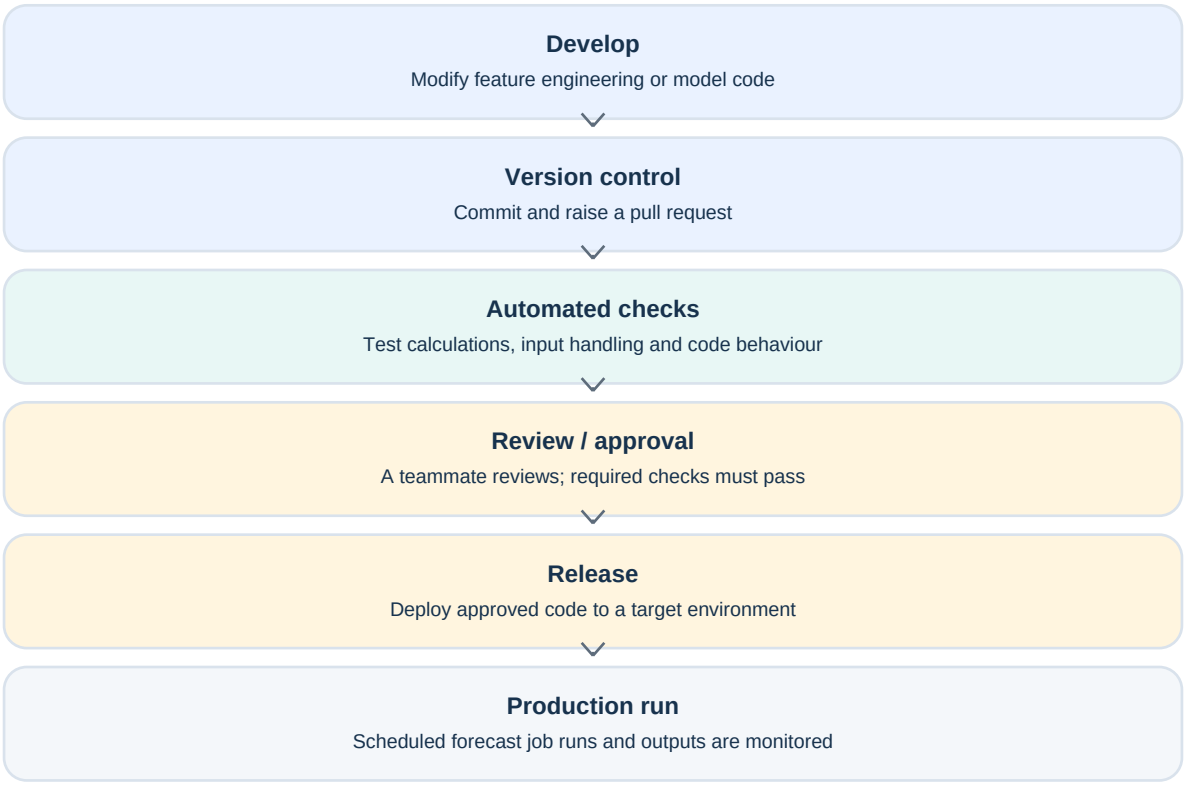
Minimal CI example

```yaml
name: Test Forecasting Code

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: pytest
```

This workflow checks out the repository, installs dependencies and runs tests. It is CI, not a complete CI/CD pipeline, because it does not deploy the code.

3. CI/CD in a data science workflow

Imagine you change feature engineering logic in a retail demand forecasting pipeline. A sensible process might look like this:



What to know as a data scientist

Priority	What you should understand
Essential	CI vs CD; Git branches and pull requests; automated tests; what a workflow file does; why checks happen before deployment.
Good working knowledge	Write a basic workflow; run pytest; manage dependencies and secrets; understand dev, test/staging and production environments.
MLOps specialisation	Enterprise deployment infrastructure, complex cloud permissions, containers/orchestration, rollback strategies, model serving and advanced monitoring.

Important for ML

Deploying code is not the same as training or deploying a model. Model training, performance validation, model registry updates and monitoring may be separate steps in an MLOps workflow.

Interview-ready explanation

Suggested answer

“CI/CD automates the process of validating code changes and releasing them reliably. CI runs checks and tests on changes; Continuous Delivery keeps validated code ready for release, while Continuous Deployment releases it automatically when the required checks pass. GitHub Actions can orchestrate these steps through YAML workflows. In data science, it can test feature engineering and model code and help deploy changes to production pipelines.”

4. Pydantic: the high-level idea

Pydantic is a Python library for defining data structures and validating/parsing data against them. You describe the fields and types you expect; Pydantic checks incoming data, converts compatible values where appropriate, and reports validation errors when the data does not meet the schema.

> **Simple mental model**
> Pydantic is like a clearly defined contract for the data entering or moving through your Python application.

Four common uses

Use	What it does	Retail/data-science example
Configuration / settings	Load and validate app configuration from supported sources such as environment variables.	Check required settings such as bucket name, API endpoint and credentials.
Data validation / parsing	Check required fields, types and constraints; parse compatible values.	Validate that a price is positive and stock count is non-negative.
API development	Validate request data and shape application responses; commonly used with FastAPI.	Accept a forecast request with product ID and horizon.
LLM structured output	Define the expected shape of an LLM result and validate the returned fields.	Extract product name, discount and stock status into structured fields.

Example: validate product data

```
from pydantic import BaseModel, Field

class Product(BaseModel):
    product_id: str
    price: float = Field(gt=0)
    stock: int = Field(ge=0)

product = Product(
    product_id="SKU123",
    price="19.99",
    stock=25
)

print(product.price)       # 19.99
print(product.model_dump())
```

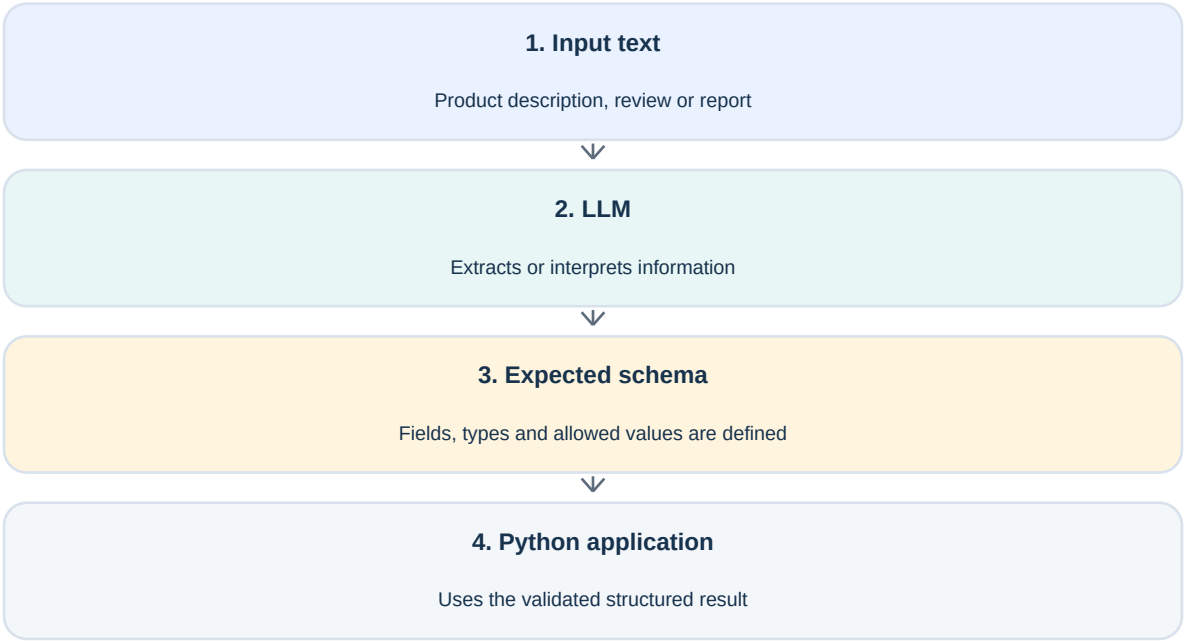
- `product_id` is expected to be a string.
- `price` must be greater than zero; the compatible string `"19.99"` can be parsed as a float.
- `stock` must be a non-negative integer. Invalid values raise a validation error.

Useful methods to recognise

Method / feature	Purpose
BaseModel	Define a model/schema with fields and types.
Field(...)	Add constraints or metadata, such as greater-than-zero.
model_validate(...)	Validate data such as a dictionary against a model.
model_dump()	Convert a validated model to a Python dictionary.
Pydantic Settings	Manage typed application settings and configuration.

5. Why Pydantic is used with LLMs

LLMs generate language, but an application often needs predictable fields that code can use. Pydantic can define the schema for the desired response. An LLM API or framework that supports structured outputs can use that schema to constrain or parse the result.



Example: define an extraction schema

```
from typing import Literal
from pydantic import BaseModel

class ProductAnalysis(BaseModel):
    product_name: str
    discount_percent: float
    stock_status: Literal[
        "in_stock", "out_of_stock", "unknown"
    ]
```

For example, an LLM might return this structured result:

```
{
  "product_name": "Blue Jacket",
  "discount_percent": 30,
  "stock_status": "in_stock"
}
```

Important limitation

Pydantic validates structure and types; it does not guarantee that the LLM's claims are factually correct. Validate business rules and verify important facts separately. Structured-output support depends on the LLM API or framework being used.

Practical applications for retail

- Extract product attributes from unstructured descriptions.
- Convert promotion text into fields such as discount percentage, start date and eligibility.
- Classify customer feedback into a defined set of categories.
- Turn competitor observations into consistent records that can be analysed with pandas or stored in a database.

What to learn first

Learn	Why
BaseModel and Field	Define fields, types and constraints.
model_validate() and model_dump()	Validate incoming data and convert it to a dictionary.
Structured LLM outputs	Use a schema to request and consume predictable LLM results.

6. Quick recap & interview checklist

Question	Short answer
What is CI/CD?	A set of practices that automates code validation and release.
What is CI?	Automatically build and test code changes.
What is CD?	Prepare changes for release or automatically deploy them, depending on the setup.
Is a scheduled GitHub Actions job CI/CD?	Not by itself. Scheduling is automation; CI/CD adds automated validation and/or release steps.
What is GitHub Actions?	A tool that runs workflows defined in YAML when triggers occur.
What is Pydantic?	A Python library for defining data schemas and validating/parsing data.
Why use Pydantic with an LLM?	To define and validate the expected structure of generated output.
Does Pydantic guarantee an LLM answer is true?	No. It checks structure and constraints, not factual correctness.

Before an interview, make sure you can explain…

- CI vs Continuous Delivery vs Continuous Deployment.
- How a pull request can trigger tests in GitHub Actions.
- Why scheduling a Python script is different from CI/CD.
- How Pydantic validates a dictionary or API request.
- How a Pydantic schema helps an LLM return structured fields.

Final takeaway

CI/CD makes code changes easier to test and release reliably. **Pydantic** makes data structures and inputs more predictable. Both help build robust Python applications and production data workflows.

Further reading

Pydantic documentation: https://docs.pydantic.dev/ • GitHub Actions documentation: https://docs.github.com/actions • OpenAI Structured Outputs guide: https://platform.openai.com/docs/guides/structured-outputs